

CSE 221: Algorithms

Chapter 4 — Sorting Algorithms

Prepared by Ariyan Hossain [AYO]

Sorting rearranges a collection into a desired order (typically ascending). This chapter covers **Merge Sort**, **Quick Sort**, and **Counting Sort**.

1 Merge Sort

Merge sort is a classic divide-and-conquer algorithm:

- **Divide:** split the array into two halves,
- **Conquer:** recursively sort each half,
- **Combine:** merge the two sorted halves.

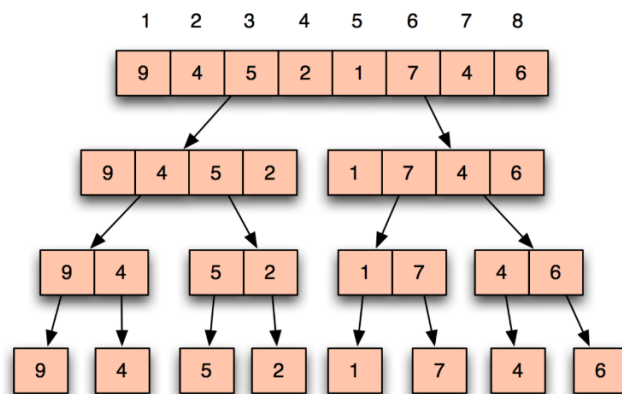


Illustration of "Divide" of Merge Sort

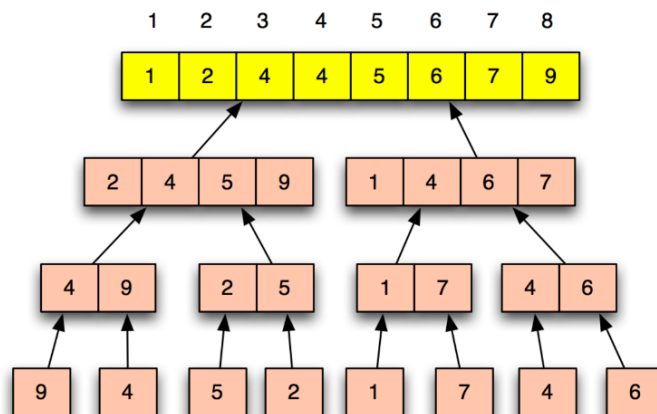


Illustration of "Conquer & Combine" of Merge Sort

1.1 Merge operation

To merge two sorted arrays, keep pointers at the front of each array and repeatedly take the smaller element.

1.1.1 Simulation (Merge Operation)

Let $A = [2, 7, 13, 20]$ and $B = [1, 9, 11, 12]$. We use indices i (for A) and j (for B), and we build C of length $|A| + |B| = 8$.

Array state	Workings
$A = [\underline{2}_i, 7, 13, 20]$ $B = [1, \underline{9}_j, 11, 12]$ $C = [_, _, _, _, _, _, _, _]$	$i = 0, j = 0$ Compare $A[i] = 2$ and $B[j] = 1$ $2 \not< 1$; set $C[0] = 1$; $j \leftarrow 1$
$A = [\underline{2}_i, 7, 13, 20]$ $B = [1, \underline{9}_j, 11, 12]$ $C = [1, _, _, _, _, _, _, _]$	$i = 0, j = 1$ Compare $A[i] = 2$ and $B[j] = 9$ $2 < 9$; set $C[1] = 2$; $i \leftarrow 1$
$A = [2, \underline{7}_i, 13, 20]$ $B = [1, \underline{9}_j, 11, 12]$ $C = [1, 2, _, _, _, _, _, _]$	$i = 1, j = 1$ Compare $A[i] = 7$ and $B[j] = 9$ $7 < 9$; set $C[2] = 7$; $i \leftarrow 2$
$A = [2, 7, \underline{13}_i, 20]$ $B = [1, \underline{9}_j, 11, 12]$ $C = [1, 2, 7, _, _, _, _, _]$	$i = 2, j = 1$ Compare $A[i] = 13$ and $B[j] = 9$ $13 \not< 9$; set $C[3] = 9$; $j \leftarrow 2$
$A = [2, 7, \underline{13}_i, 20]$ $B = [1, 9, \underline{11}_j, 12]$ $C = [1, 2, 7, 9, _, _, _, _]$	$i = 2, j = 2$ Compare $A[i] = 13$ and $B[j] = 11$ $13 \not< 11$; set $C[4] = 11$; $j \leftarrow 3$
$A = [2, 7, \underline{13}_i, 20]$ $B = [1, 9, 11, \underline{12}_j]$ $C = [1, 2, 7, 9, 11, _, _, _]$	$i = 2, j = 3$ Compare $A[i] = 13$ and $B[j] = 12$ $13 \not< 12$; set $C[5] = 12$; $j \leftarrow 4$
$A = [2, 7, \underline{13}_i, 20]$ $B = [1, 9, 11, 12]$ $C = [1, 2, 7, 9, 11, 12, _, _]$	$i = 2, j = 4$ ($j \geq \text{len}(B)$) Append remaining $A[i..] = [13, 20]$ into C ; set $C[6] = 13$; $i \leftarrow 3$ and $C[7] = 20$; $i \leftarrow 4$
$A = [2, 7, 13, 20]$ $B = [1, 9, 11, 12]$ $C = [1, 2, 7, 9, 11, 12, 13, 20]$	Final merged output.

Merge simulation of $A = [2, 7, 13, 20]$ and $B = [1, 9, 11, 12]$

1.2 Pseudocode (Merge Sort)

Merge Sort of array A

```

MERGE-SORT(A)
  if length(A) = 1
    return A
  else
    A1 ← MERGE-SORT(A[1 : n/2])
    A2 ← MERGE-SORT(A[n/2 + 1 : n])
    A ← MERGE(A1, A2)
    return A

MERGE(A, B)
  i ← 0, j ← 0
  C ← empty list
  while i < length(A) and j < length(B)
    if A[i] ≤ B[j]
      add A[i] to C
      i ← i + 1
    else
      add B[j] to C
      j ← j + 1
  add remaining elements of A to C
  add remaining elements of B to C
  return C

```

Advantages of Merge Sort:

- Consistent time complexity $\mathcal{O}(n \log n)$, making it efficient for large datasets.
- Stable sorting algorithm that preserves the relative order of equal elements.

Disadvantages of Merge Sort:

- Requires additional memory space for temporary arrays, leading to space complexity $\mathcal{O}(n)$.
- Slower for small datasets compared to simpler algorithms such as insertion sort.

Follow-up Questions

Q) What is the time complexity of Merge operation?

Ans: $\mathcal{O}(n + m)$

Q) What is the recurrence equation and time complexity of Merge Sort?

Ans: It depends on the number of subproblems, each subproblem size, and the work done at each step. In each step, there is 2 subproblem where size gets divided by 2 ($n/2$) and work done is n at each step.

$$\begin{aligned}
 T(n) &= 2T\left(\frac{n}{2}\right) + n \\
 &= \mathcal{O}(n \log_2 n)
 \end{aligned}$$

2 Quick Sort

Quick sort is a divide-and-conquer algorithm:

- **Divide:** partition the array around a pivot,
- **Conquer:** recursively sort the left and right parts,
- **Combine:** since the pivot ends in its final position after partitioning, no merging is needed.

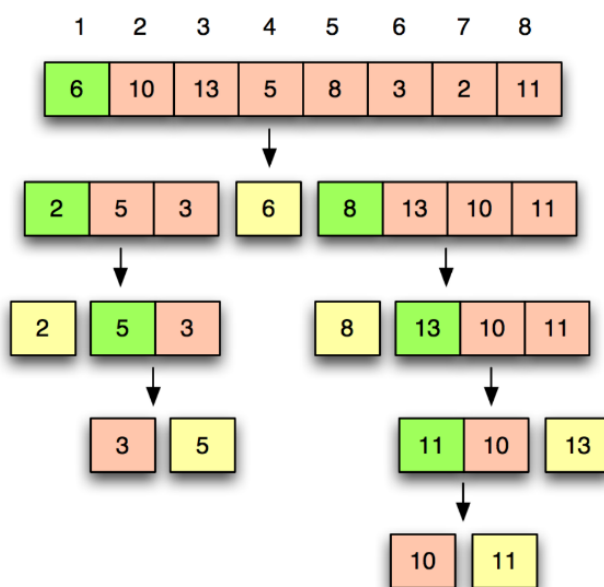


Illustration of "Divide" of Quick Sort

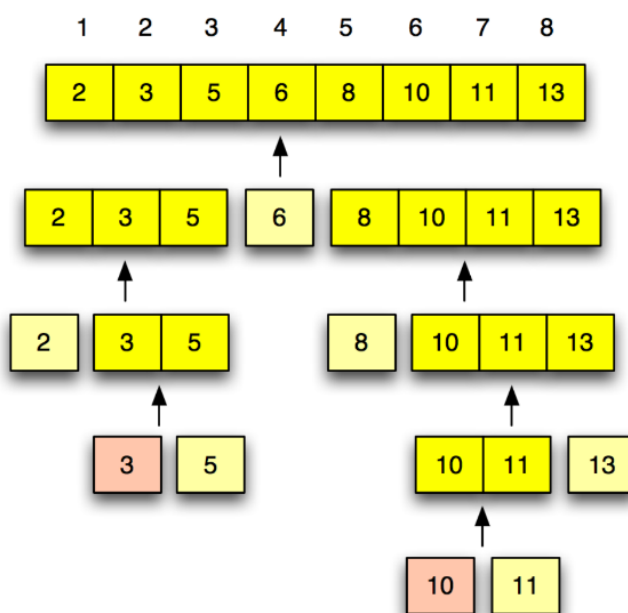


Illustration of "Conquer & Combine" of Quick Sort

2.1 Partition operation

Partitioning rearranges the array so that all elements $\leq$ pivot are on the left and all elements $\geq$ pivot are on the right.

2.1.1 Simulation (Partition Operation)

Let $A = [6, 10, 13, 5, 8, 3, 2, 11]$ and pivot=6. We use indices i and j for iterating and swapping, and l and r for indicating first (and also pivot) and last element.

Array state	Workings
$A = [\bar{6}_{i=l}, \underline{10}_j, 13, 5, 8, 3, 2, 11_r]$	$l = 0, i = 0, j = 1$ Compare $A[j] = 10$ and $A[l] = 6$ $10 > 6; j \leftarrow 2$
$A = [\bar{6}_{i=l}, 10, \underline{13}_j, 5, 8, 3, 2, 11_r]$	$l = 0, i = 0, j = 2$ Compare $A[j] = 13$ and $A[l] = 6$ $13 > 6; j \leftarrow 3$
$A = [\bar{6}_{i=l}, 10, 13, \underline{5}_j, 8, 3, 2, 11_r]$	$l = 0, i = 0, j = 3$ Compare $A[j] = 5$ and $A[l] = 6$ $5 < 6; i \leftarrow 1; \text{ swap } A[i] \text{ and } A[j]; j \leftarrow 4$
$A = [6_l, \bar{5}_i, 13, 10, \underline{8}_j, 3, 2, 11_r]$	$l = 0, i = 1, j = 4$ Compare $A[j] = 8$ and $A[l] = 6$ $8 > 6; j \leftarrow 5$
$A = [6_l, \bar{5}_i, 13, 10, 8, \underline{3}_j, 2, 11_r]$	$l = 0, i = 1, j = 5$ Compare $A[j] = 3$ and $A[l] = 6$ $3 < 6; i \leftarrow 2; \text{ swap } A[i] \text{ and } A[j]; j \leftarrow 6$
$A = [6_l, 5, \bar{3}_i, 10, 8, 13, \underline{2}_j, 11_r]$	$l = 0, i = 2, j = 6$ Compare $A[j] = 2$ and $A[l] = 6$ $2 < 6; i \leftarrow 3; \text{ swap } A[i] \text{ and } A[j]; j \leftarrow 7$
$A = [6_l, 5, 3, \bar{2}_i, 8, 13, 10, \underline{11}_{j=r}]$	$l = 0, i = 3, j = 7$ Compare $A[j] = 11$ and $A[l] = 6$ $11 > 6; j \leftarrow 8$
$A = [6_l, 5, 3, \bar{2}_i, 8, 13, 10, 11_r]$	$l = 0, i = 3, j = 8, r = 7$ Since $j > r; \text{ swap } A[i] \text{ and } A[l]$
$A = [2_l, 5, 3, \bar{6}_i, 8, 13, 10, 11_r]$	return $i = 3$

Partition simulation of $A = [6, 10, 13, 5, 8, 3, 2, 11]$ using first element as pivot

2.2 Pseudocode (Quick Sort)

Quick Sort of array A (Partition = Lomuto style)

```

QUICKSORT(A, l, r)
  if l < r
    p ← PARTITION(A, l, r)
    QUICKSORT(A, l, p - 1)
    QUICKSORT(A, p + 1, r)

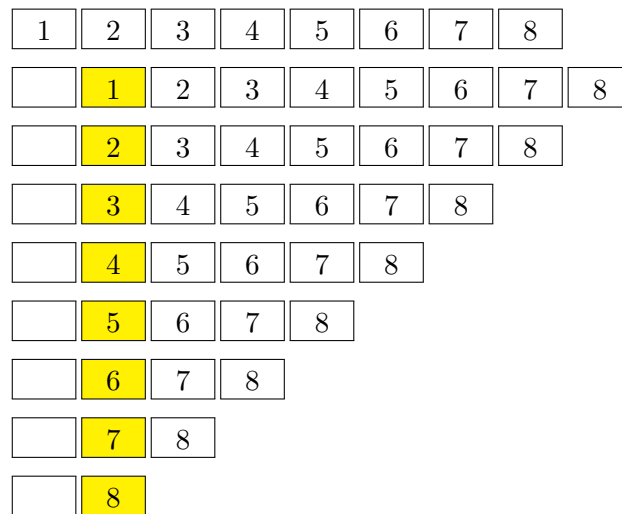
PARTITION(A, l, r)
  i ← l
  for j ← l + 1 to r; j++
    if A[j] ≤ A[l]
      i ← i + 1
      swap A[i] and A[j]
  swap A[l] and A[i]
  return i

```

2.3 Worst Case vs Best Case of Quick Sort

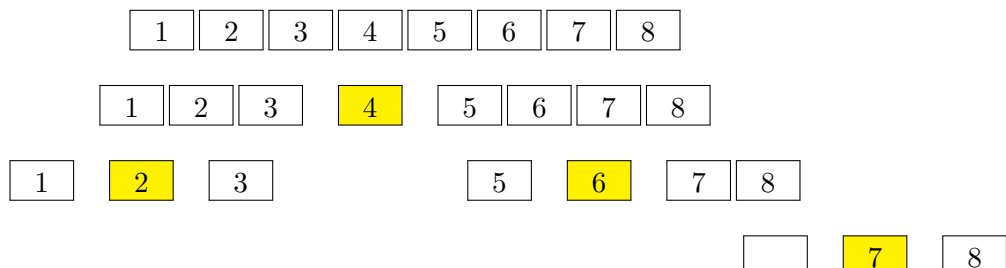
2.3.1 Worst Case

Worst Case happens when the pivot is the first or last element of a sorted (ascending or descending) array. The result is that one of the partitions is always empty.



2.3.2 Best Case

Best case happens when the pivot divides the array into two nearly equal halves, resulting in balanced partitions



Advantages of Quick Sort:

- Fast in practice with average time complexity $\mathcal{O}(n \log n)$, making it efficient for large datasets.
- Does not require additional memory space since it is an in-place sorting algorithm.

Disadvantages of Quick Sort:

- Worst-case time complexity can degrade to $\mathcal{O}(n^2)$ when partitions are highly unbalanced.
- Unstable sorting algorithm, not preserving the relative order of equal elements

Follow-up Questions

Q) If Quick Sort is $\mathcal{O}(n \log n)$ on average, but Merge Sort is $\mathcal{O}(n \log n)$ always, why not use Merge Sort? Isn't it faster?

Ans: Quick Sort usually has a smaller constant factor (c) than that of Merge Sort and often runs faster in practice because it hits the average case much more often than the worst case.

Q) What is the recurrence equation and time complexity of Quick Sort in the worst case?

Ans: It depends on the number of subproblems, each subproblem size, and the work done at each step. In each step, there is 1 subproblem where size gets subtracted by 1 ($n - 1$) and work done is n at each step.

$$\begin{aligned} T(n) &= T(n - 1) + n \\ &= \mathcal{O}(n^2) \end{aligned}$$

Q) What is the recurrence equation and time complexity in the best case?

Ans: It depends on the number of subproblems, each subproblem size, and the work done at each step. In each step, there is 2 subproblem where size gets divided by 2 ($n/2$) and work done is n at each step.

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \\ &= \mathcal{O}(n \log_2 n) \end{aligned}$$

Q) How do you avoid the worst case when choosing the first element as pivot?

Ans: Use **Randomized Quicksort**: swap the pivot with a random element in the subarray before partitioning. This makes the expected running time $\mathcal{O}(n \log n)$.

3 Counting Sort

Counting sort is a **non-comparative** sorting algorithm for integers in a limited range. If values lie in $[0, k - 1]$, we count frequencies and then use prefix sums to place elements. It is efficient when the range of the input data, k , is not significantly greater than the number of elements to be sorted.

3.1 Simulation (Counting Sort)

Step 1: Find out the range k ($\max+1$) from the given array.

	0	1	2	3	4	5	6	7
inputArray	2	5	3	0	2	3	0	3

Step 2: Initialize a `countArray[]` of length k ($5+1$) with all elements as 0. This array will be used to store the occurrences of the elements of the input array.

	0	1	2	3	4	5
countArray	0	0	0	0	0	0

Step 3: In the `countArray[]`, store the count of each unique element of the input array at their respective indices.

	0	1	2	3	4	5
countArray	2	0	2	3	0	1

Step 4: Store the cumulative sum of the elements of the `countArray[]` by doing `countArray[i] = countArray[i - 1] + countArray[i]`.

	0	1	2	3	4	5
countArray	2	2	4	7	7	8

Step 5: Scan the input array from right to left (to keep it stable). Place each value in the output array using the `countArray[]` index, then decrement its count (to deal with duplicates).

For $i = 7$,

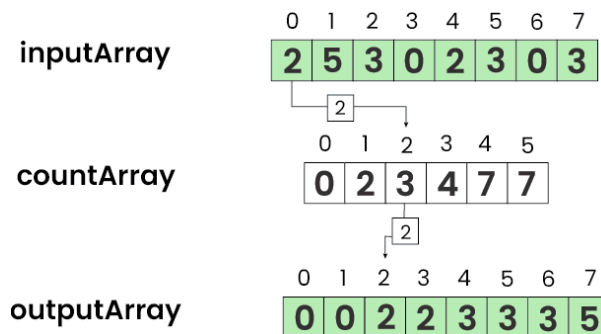
inputArray	0	1	2	3	4	5	6	7
	2	5	3	0	2	3	0	3
						3		
countArray	0	1	2	3	4	5		
	2	2	4	7	7	8		
						7-1=6		
outputArray	0	1	2	3	4	5	6	7
							3	

For $i = 6$,

inputArray	0	1	2	3	4	5	6	7
	2	5	3	0	2	3	0	3
						0		
countArray	0	1	2	3	4	5		
	2	2	4	6	7	8		
		1						
outputArray	0	1	2	3	4	5	6	7
		0					3	

⋮

For $i = 0$,



3.2 Pseudocode (Counting Sort)

Counting Sort

```
COUNTING-SORT(A, B, k)

    // C[0..k] stores counts
    for i ← 0 to k
        C[i] ← 0

    // Count each value in A
    for j ← 0 to length(A) - 1
        C[A[j]] ← C[A[j]] + 1

    // Prefix sums
    for i ← 1 to k
        C[i] ← C[i] + C[i - 1]

    // Build output array B (stable)
    for j ← length(A) - 1 downto 0
        B[C[A[j]] - 1] ← A[j]
        C[A[j]] ← C[A[j]] - 1
```

Advantages of Counting Sort:

- Consistent time complexity $\mathcal{O}(n + k)$, making it efficient when the range k is not too large.
- Stable sorting algorithm that preserves the relative order of equal elements.

Disadvantages of Counting Sort:

- Inefficient when the range of values k is very large.
- Requires extra memory for **countArray** and **outputArray**, leading to space complexity $\mathcal{O}(n + k)$.
- Does not directly work for decimal values or negative values.

Follow-up Questions

Q) What is the time complexity of Counting Sort?

Ans: The first loop takes $\mathcal{O}(k)$, the second loop takes $\mathcal{O}(n)$, the third loop takes $\mathcal{O}(k)$, and the last loop takes $\mathcal{O}(n)$. Hence, the overall time complexity is $\mathcal{O}(n + k)$.

Q) How to work with negatives, decimals or numbers starting from a large value?

Ans: Negative numbers can be handled by shifting all values to make them non-negative, decimals can be converted into integers by multiplying with a suitable power of 10, and numbers starting from a high value can be scaled down by subtracting the minimum value.